UNIONS

Unions

- A union is a derived data type like a structure with members that share the same storage space

```
struct student {
    int age;
    char grade;
};
```

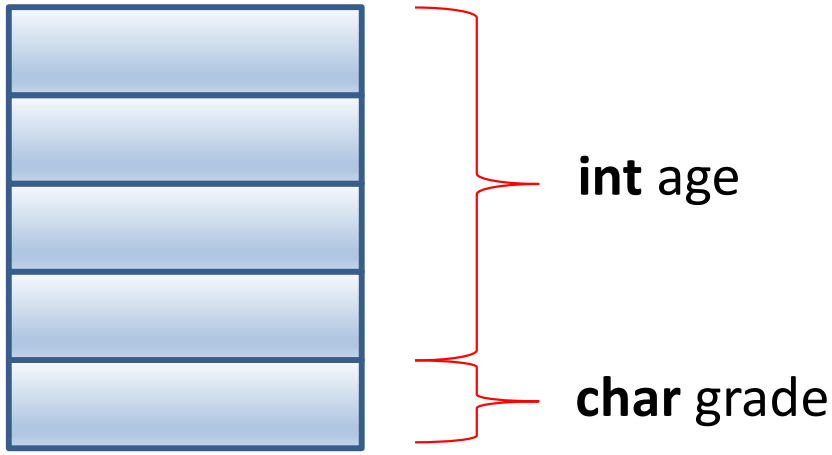
```
union charater {
    int value;
    char letter;
};
```

Unions

- A union is a derived data type like a structure with members that share the same storage space

```
struct student {
    int age;
    char grade;
};
```

```
union charater {
    int value;
    char letter;
};
```

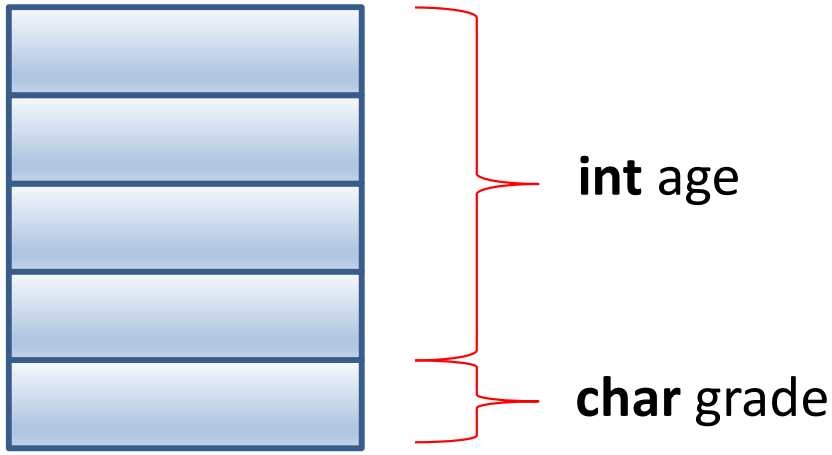


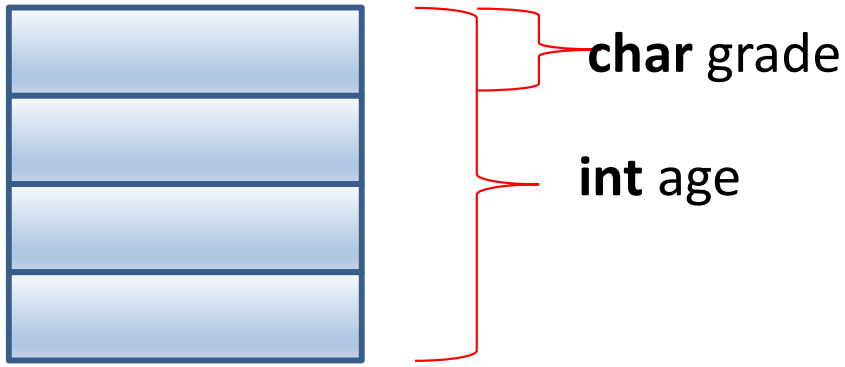
Unions

- A union is a derived data type like a structure with members that share the same storage space

```
struct student {
    int age;
    char grade;
};
```

```
union charater {
    int value;
    char letter;
};
```





Union: Share same Memory Location

```
struct student {
    int age;
    char grade;
};

union charater {
    int value;
    char letter;
};

void main () {
    struct student st1;
    union charater mychar;
    printf("Memory address of st1.age: %p\n", &st1.age);
    printf("Memory address of st1.grade: %p\n", &st1.grade);
    printf("\n");
    printf("Memory address of mychar.value: %p\n", &mychar.value);
    printf("Memory address of mychar.letter: %p\n", &mychar.letter);
}
```

Union: Share same Memory Location

```c
struct student {
    int age;
    char grade;
};

union charater {
    int value;
    char letter;
};

void main () {
    struct student st1;
    union charater mychar;
    printf("Memory address of st1.age: %p\n", &st1.age);
    printf("Memory address of st1.grade: %p\n", &st1.grade);
    printf("\n");
    printf("Memory address of mychar.value: %p\n", &mychar.value);
    printf("Memory address of mychar.letter: %p\n", &mychar.letter);
}
```

```
Memory address of st1.age: 000000000062FE10
Memory address of st1.grade: 000000000062FE14

Memory address of mychar.value: 000000000062FE00
Memory address of mychar.letter: 000000000062FE00
```

Unions

- The members of a union can be of any data type
- The number of bytes used to store a union must be at least enough to hold the largest member
  - So, size of the union is the size of its largest element

Unions: SIZE

```
struct student {
    int age;
    char grade;
    double gpa;
};

union charater {
    int value;
    char letter;
    double points;
};

void main () {
    struct student st1;
    union charater mychar;
    printf("size of st1: %d\n", sizeof(st1));
    printf("size of mychar: %d\n", sizeof(mychar));

}
```

Unions: SIZE

```c
struct student {
    int age;
    char grade;
    double gpa;
};

union charater {
    int value;
    char letter;
    double points;
};

void main () {
    struct student st1;
    union charater mychar;
    printf("size of st1: %d\n", sizeof(st1));
    printf("size of mychar: %d\n", sizeof(mychar));

}
```

```
size of st1: 16
size of mychar: 8
```

Union: Example

```
union charater {
    int value;
    char letter;
    double points;
};

void main () {
    union charater mychar;
    mychar.value = 65;
    printf("mychar.value %d\n", mychar.value);
    printf("mychar.letter: %c\n", mychar.letter);
    printf("mychar.points: %f\n", mychar.points);

}
```

Union: Example

```
union charater {
    int value;
    char letter;
    double points;
};

void main () {
    union charater mychar;
    mychar.value = 65;
    printf("mychar.value %d\n", mychar.value);
    printf("mychar.letter: %c\n", mychar.letter);
    printf("mychar.points: %f\n", mychar.points);

}
```

```
mychar.value 65
mychar.letter: A
mychar.points: 0.000000
```

Operators on Unions

- The operations that can be performed on a union are the following:
  - **assigning** a union to another union of the **same type**
  - **taking the address** (&) of a union variable
  - accessing union members using the structure **member operator** and the structure **pointer operator**

- Unions **may not be compared using operators == and !=** same as the structures

Unions

- In a declaration, a union may be initialized with a value of the same type as **the first union member**

```
union charater {
    int value;
    char letter;
};

void main () {
    union charater mychar = {68};
    printf("mychar.value %d\n", mychar.value);
    printf("mychar.letter: %c\n", mychar.letter);
}
```

Exercise:

- A shop has following two items for sale.
  - Books
  - Cakes
- Each of the items store the details as given below
  - Books: id, title, author, price
  - Cakes: flavour, weight, price
- What is the most convenient way to store data?

Alternative 1 (Using structures)

```
struct item{
        int id;
        char *title;
        char *author;
        double price;
        int flavour;
        int weight;
}
```

Alternative 1 (Using structures)

```
int main(){
        struct item cake;
        cake.flavour = 1;
        cake.weight = 400;
        cake.price = 1500;
return 0;
}
```

To store details of a cake item

Alternative 1 (Using structures)

```
int main(){
        struct item cake;
        cake.flavour = 1;
        cake.weight = 400;
        cake.price = 1500;
return 0;
}
```

```
struct item{
    int id;
    char *title;
    char *author;
    double price;
    int flavour;
    int weight;
}
```

Unused ?

Alternative 1 (Using structures)

When we need to store details about a book, space allocated to the other data members remain unused.

Wasting memory!

```
struct item{
    int id;
    char *title;
    char *author;
    double price;
    int flavour;
    int weight;
}
```

Alternative 1 (Using structures)

```
int main(){
struct item cake;
printf("%d bytes", sizeof(cake));
return 0;
}
```

36 bytes

```
struct item{
    int id;
    char *title;
    char *author;
    double price;
    int flavour;
    int weight;
}
```

Alternative 1 (Using structures)

```
struct item{
        int id;
        char *title;
        char *author;
        double price;
        int flavour;
        int weight;
}
```

int id;	4 bytes
char *title;	8 bytes
char *author;	8 bytes
double price;	8 bytes
int flavour;	4 bytes
int weight;	4 bytes

Alternative 2 (Using unions)

```
struct item{
        double price;
        union {
                struct{
                        int id;
                        char *title;
                        char *author;
                } book;
                struct{
                        int flavour;
                        int weight;
                } cake;
        }product;
}
```

Alternative 2 (Using unions)

```
Int main(){
        struct item itm;

        itm.product.cake.flavour = 1;
        itm.product.cake.weight = 400;

        printf(“ Flavour : %d \n”, itm.product.cake.flavour);
}
```